Artificial Intelligence Laboratory 1: Agents

DT8042, Halmstad University

November 2025 (Niclas)

Introduction

This lab is intended to allow you to practice designing and implementing intelligent agents in two application domains: (*i*) Mobile robot: collecting energy blocks and (*ii*) Simplified poker game: winning more games and money. You will implement four (very simple) agents:

- **Random agent**: acts randomly, disregarding any external input.
- **Fixed agent**: acts based on a sequence of pre-specified actions, disregarding any external input.
- **Reflex agent**: acts based on the agent's current perception of the world, but not based on past perceptions.
- **Agent with memory**: acts based on the agent's current perception of the world and past perceptions.

Task 1: Mobile robot exploration

You will program a mobile robot, called "Pioneer P3-DX", to explore a maze-like environment (see Figure 1). This robot has two wheels, driven by independent motors. It is equipped with 16 ultrasonic sensors and an "energy sensor" (which provides the relative position to the nearest red block). Your task is to implement different kinds of agents, each trying to collect energy (red block) from the environment. If you feel ambitious, you can also try to ensure that the robot doesn't crash into walls, doesn't aimlessly wander in circles, etc. – neither of those things is required, though.

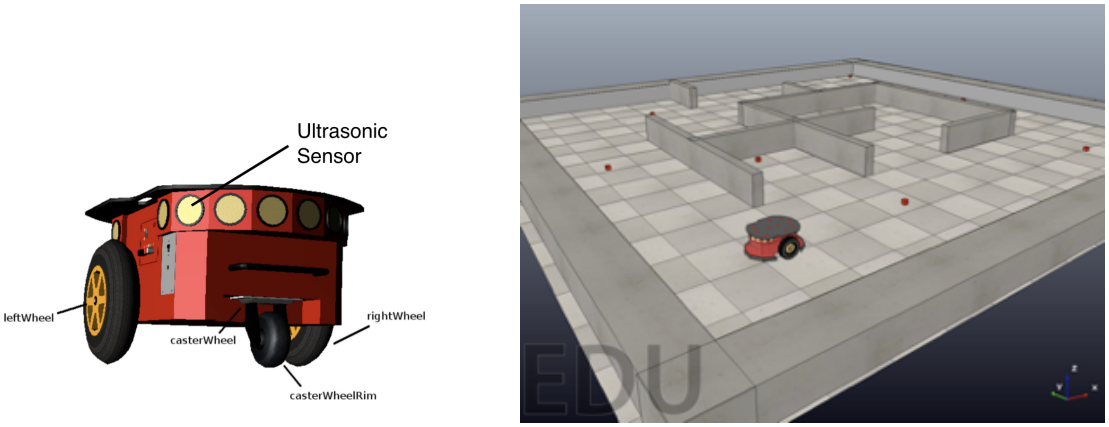

Figure 1: (left) Pioneer P3-DX; (right) a simple maze-like environment

Environment Setup

The following software tools are required for Lab 1. Make sure you have installed them and verified that they work properly.

- CoppeliaSim/V-REP: a virtual robot simulation platform (choose CoppeliaSim V4.7.0 rev4, the free educational version, e.g. "Windows binary package").
- Python: general-purpose, high-level programming language. If you do not have experience with Python programming, we suggest the following two resources:
  - Codecademy: detailed python tutorial with the web-integrated python interpreter
  - Pycharm: an intelligent python integrated development environment.
  - Jupyter Notebook: a browser-based interpreter that allows you to interactively work with Python. You can install Anaconda, an open-source distribution of the Python, which provides Jupyter Notebook as well.

In the **Lab1.zip** file, you can find the following resources:

- **Lab1-Agents-Task1-World.ttt** file is a V-REP scene file. Load it in V-REP and try to get familiar with the GUI. Several objects are defined in this scene. The main objects are the 'Pioneer' robot, 12 red blocks (objects to collect), outer and inner walls. Click items within the 'Scene hierarchy' tree to inspect various objects and their properties.
  - The 'Pioneer' robot is set up to be controlled remotely by an external controller script.
- **Lab1_Agents_Task1_Pioneer.py** file is the remote python client that can act as the controller of the 'Pioneer' robot. In order for the script to work, three files from the V- REP's remoteAPI package need to be in the same folder as **Lab1_Agents_Task1_Pioneer.py**:
  - **sim.py** and **simConst.py** from the directory (the unzipped folder of the Windows binary package download):

    `<PATH_TO_V-REP>\programming\legacyRemoteApi\remoteApiBindings\python\python`
  - the appropriate remote API library, depending on your system **remoteApi.dll** (Windows), **remoteApi.dylib** (Mac) or **remoteApi.so** (Linux) from

    `<PATH_TO_V-REP> \programming\legacyRemoteApi\remoteApiBindings\lib\lib`

  Run the simulation, by first clicking start simulation in V-REP and then running client script: **Lab1_Agents_Task1_Pioneer.py**
- Take a look at the code within **Lab1_Agents_Task1_Pioneer.py** and make sure you understand what it is doing. Figure out the following (at least on a conceptual level):
  - Which two sensors of the robot have been used?
  - How to control the motion of the robot?
  - How to assign different speeds to the two wheels?
  - How to get the nearest block's position? And how to collect it?
- Make sure you understand the following functions you will be using:
  - World.getSensoreReading(. . .)
  - World.setMotorSpeeds(. . .)
  - World.collectNearestBlock(...)
  - World.execute(. . .)

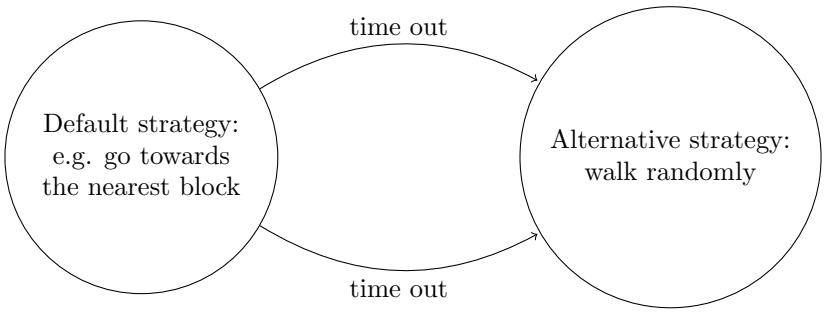

Figure 2: Example Strategy

Tasks

1a. Implement an agent that takes random actions.

1b. Implement an agent that performs exactly the same sequence of actions repeatedly, ignoring sensor input. Try to collect at least 1-2 red blocks.

1c. Implement a simple reflex agent that takes sensor input and makes decisions based on the current sensor readings (no past information should be used).

1d. Implement an agent that remembers (some) past sensor readings and actions taken and utilizes this information to reach the goal in a more intelligent way. One type of problem you probably encounter with the reflex agent is that it gets stuck at some part of the maze (e.g. in a corner). A very simple way to deal with this is to use a time counter for reaching the next block's position. If the robot is taking too much time to reach one of the blocks, it probably got stuck somewhere, and therefore it should change its strategy to deal with this (see Figure 2 as an example).

Please note that it is not required to collect all the blocks in the maze, i.e., "collecting as many blocks as possible" is not the core focus of these tasks. It is advised to distribute your time wisely on each task of this lab.

Task 2: Poker game agent

Introduction

This part of the lab requires you to implement three different agents to play a simplified poker game, the rules of the game are stated as follows:

- There will be two agents playing against each other within the game.
- We assume that coins/money are provided by the "central bank", agents don't play with their own money and they have an unlimited amount of money to play. The goal for each agent is to win as much money as possible while making the opponent win as little as possible. In other words, the agents should be evaluated based on the *difference* between their winnings and the winnings of their opponents.
- There are 52 cards in a deck, divided into four suits (i.e. spade, heart, club, and diamond) of 13 ranks each (i.e. Ace, K, Q, J, and from 2 to 10). The suits are all of equal value, i.e., no suit is higher than any other suit.
- This is a simplified version of a poker game and agents are only dealing with hands of three cards. The possible hands are "three of a kind", "a pair" and the rest are "high cards". Each of these three types has a maximum of 13 ranks, corresponding to the number of variants in each

suit. Therefore, one way to evaluate the hand is to assign a score, according to the type and the strength. For example, high cards can be assigned with scores ranging from 1 to 13; paris from 14 to 26; three-of-a-kind from 27 to 39.

- There will be 50 hands for each game.
- Each hand includes the following flow:
  - **Card dealing phase**: assign a randomly generated hand (three cards) to each agent.
  - **Bidding phase (1-3)**: agents decide how much money ($0-50) they want to bid into the pot. There are three bidding phases for every hand.
  - **Showdown phase**: after three bidding phases, both agents show their hands, and the agent with a stronger hand gets the pot.
  - After every 50 hands compute the difference in winnings between the two agents.

Here is an example game flow of one hand:

- **Card dealing phase**, generate a random hand (3 cards) for each agent:
  Agent 1 got a hand '4h, Ks, Kc'
  Agent 2 got a hand '5s, 5h 5c'
- **Bidding phase 1**, two agents decide the amount of money to bid:
  Agent 1 bids $20
  Agent 2 bids $30
- **Bidding phase 2**:
  Agent 1 bids $30
  Agent 2 bids $25
- **Bidding phase 3**:
  Agent 1 bids $5
  Agent 2 bids $45
- **Showdown phase**, both agents show their hand. Agent 2 has a 'three of a kind' while Agent 1 has only a pair of King. Therefore, agent 2 wins and gets the pot, which is $155.

Tasks

2a. Implement a random agent that bids randomly.

2b. Implement a fixed agent.

2c. Build the environment of the game, and have the two agents play against each other. At this point, you need to implement the following:

(a) game flow, according to the rules above.

(b) hand identification and strength evaluation function for this simplified poker game. **Lab1.zip** contains a simple example that checks whether there is one pair in the hand.

(c) sensor input for the agents:

i. agent's own hand (during **card dealing phase**)

ii. hand of opponent (during **showdown phase**)

iii. amount of money both agents bid (during **betting phase**).

(d) a way of recording the results.

2d. Analyse the result between a random poker agent vs. a fixed poker agent. Which agent is better? Why? To empirically back your analysis, provide metrics such as the mean and standard deviation of the bankroll difference at the end of the game for 100 games (50-hands per game).

2e. Implement a simple reflex agent and play it first against a random agent and then against a fixed agent. The reflex agent should make better betting decisions based on the strength of its own hand. Use 2d analysis tools to verify that.

Grading Criteria

- Pass: Complete all tasks (1a-d and 2a-e).
- Deadline is 2 weeks starting from the introduction session.
- Your submission should include:
  1. Code (.py or .ipynb files)
  2. A short report describing what you have done, observed, and learned. You need to specify each member's contribution (e.g. who worked on which part) if this lab was conducted in a group (of two students). You are encouraged to use the template provided.
- Bonus points: In order to be eligible to get bonus "credit" you need to complete all the mandatory tasks.

  Attempts to achieve bonus shall be explicitly indicated in the report.

  1. Your robot agent can collect at least half of the energy blocks.
  2. Your robot can avoid obstacles, i.e., it does not crash into walls.
  3. Implement a reflex agent with memory that makes betting decisions based on its current hand strength and the amount of money the opponent bet last round. Play it against the reflex agent without memroy. Is the memory agent performing better? why/ why not? what could be done to improve it?

  To get the bonus you need to complete all of the tasks above.

  Bonus can only be awarded **with the first submission**.